0040洛谷P1789 插火把

```
#include<iostream>
using namespace std;

int map[102][102];
```

先用二位数组制定一个平面空间，空间范围要适用于n为任意数的情况

注意，map[][]既不在main主函数之中，也不在任一分函数之中，是一种全局变量。全局变量会默认自动赋值为0

与之相对的是局部变量，局部变量是没有默认赋值的，需要主动int map[][]={0}

```
int main() {
    int n, m, k;
    cin >> n >> m >> k;
```

先读取n,m,k

```
    for (int i = 0; i < m;i++) {
        int x, y;
        cin >> x >> y;
```

再读取m个火把的位置

```
        for (int dx = -2;dx <= 2;dx++) {
            for (int dy = -2;dy <= 2;dy++) {
                if (abs(dx) + abs(dy) <= 2) {
```

这里用了一个非常经典的算法“曼哈顿距离”

试想：想象你只能横着走或竖着走（不能斜穿），从一点到另一点需要走多少步？

曼哈顿距离 = 横向距离 + 纵向距离，即 |x1-x2| + |y1-y2|，它衡量的是“网格街区”中的最短路径长度

菱形辐射的情况可以考虑用曼哈顿距离算法

abs()是绝对值公式，可以把负值转换为正值，源自于头文件<iostream>

```
                    int nx = x + dx;
                    int ny = y + dy;
```

在我们得到某一火把的x和y坐标后，用dx和dy表示其辐射，再用nx，ny来表示地图上所有可能会被影响到的区域

```
                    if (nx >= 1 && nx <= n && ny >= 1 && ny <= n) {
```

判断nx和ny是否会越界，若没有越界，将其标记为“1”

```
                        map[nx][ny] = 1;
                    }
                }
            }
        }
    }
    for (int i = 0; i < k; i++) {
        int x, y;
        cin >> x >> y;
```

读取k个荧石的位置

```
        for (int dx = -2; dx <= 2;dx++) {
            for (int dy = -2;dy <= 2;dy++) {
                int nx = x + dx;
                int ny = y + dy;
```

萤石是正方形辐射，我们直接围绕萤石画一个正方形范围就行了

```
                if (nx >= 1 && nx <= n && ny >= 1 && ny <= n) {
                    map[nx][ny] = 1;
                }
            }
        }
    }

    int ans = 0;
    for (int i = 1; i <= n;i++) {
        for (int j = 1;j <= n;j++) {
            if (map[i][j] == 0) {
                ans++;
```

在这个n*n的区域内，搜寻值为0的地块，统计出来就是ans啦

```
            }
        }
    }
    cout << ans << endl;
    return 0;
}
```

完整代码

```
#include<iostream>
using namespace std;

int map[102][102];

int main() {
    int n, m, k;
    cin >> n >> m >> k;

    for (int i = 0; i < m;i++) {
        int x, y;
        cin >> x >> y;

        for (int dx = -2;dx <= 2;dx++) {
            for (int dy = -2;dy <= 2;dy++) {
                if (abs(dx) + abs(dy) <= 2) {

                    int nx = x + dx;
                    int ny = y + dy;

                    if (nx >= 1 && nx <= n && ny >= 1 && ny <= n) {

                        map[nx][ny] = 1;
```

```
                    }
                }
            }
        }
    }
    for (int i = 0; i < k; i++) {
        int x, y;
        cin >> x >> y;

        for (int dx = -2; dx <= 2;dx++) {
            for (int dy = -2;dy <= 2;dy++) {
                int nx = x + dx;
                int ny = y + dy;

                if (nx >= 1 && nx <= n && ny >= 1 && ny <= n) {
                    map[nx][ny] = 1;
                }
            }
        }
    }

    int ans = 0;
    for (int i = 1; i <= n;i++) {
        for (int j = 1;j <= n;j++) {
            if (map[i][j] == 0) {
                ans++;

            }
        }
    }
    cout << ans << endl;
    return 0;
}
```